

Performance engineering

N Chapman and L G Kirby

Failure caused by poor performance renders a system just as useless as failure caused by functional errors, and can be even more expensive to correct. For many years, performance in software systems has been achieved through a ‘fix-it-later’ approach, using optimisation and tuning techniques. Recent evidence shows that, with the increased use of highly complex, multi-layered, client/server, distributed architectures, these techniques cannot be relied on to deliver even the level of performance required during early operation when workloads are small, let alone over extended periods. This paper describes some of the ways in which performance can be ‘engineered’ into systems — examples of how these techniques have been used in recent BT projects are included.

1. Introduction

1.1 Non-functionals — why bother?

It is some years since the widespread recognition that ‘testing’ should not just be considered as an activity that takes place after system and software development. It is now well embedded as a life cycle process which is more effectively described by the verification, validation and testing (VV&T) approach that all self-respecting systems developers and integrators now adopt. These days, it is not uncommon to find that the VV&T budget is 30—40% of overall project costs and, for some (typically safety-critical) systems, the percentage may be a good deal higher than this. This life cycle approach for VV&T has led to a high level of success in the delivery of ‘functionally correct’ systems, despite the added complications of distribution of both data and process, new technologies, operating systems, platforms, networks, etc.

It would make life simple if all that was required was that a system should function correctly, but, of course, developers are dealing with many ‘non-functionals’ as well — security, usability, availability, reliability, and performance are just some of these. Failure of any one of these non-functional aspects can render a system useless in just the same way that functional failure does and can be even more expensive to correct. If this is the case, one might expect that a comparable degree of importance would be accorded to the assurance of these non-functionals, to try to minimise the potentially massive costs that system failure can cause. It is well known that this does not always happen and a number of high profile failures testify to this.

There are a number of reasons why performance, in particular, is ignored during the early stages of a project. Historically, there has been a clear cycle in the approach to

performance. In the early days of computing when memory and processor power were at a premium, considerable effort was put into creating efficient systems as this was often the only way of achieving what was needed. Hardware subsequently improved and became more readily available — at that stage, the significant level of system management and system measurement tools within the mainframe environment meant that there was less need for the developers to concern themselves with performance. It was nearly always possible to tune systems to achieve the throughputs and response times necessary to meet business needs. With the arrival of highly distributed systems, significant processor power at the desktop, complex LAN and WAN configurations, ‘heterogeneous’ and ‘open’ environments, we now find ourselves at a real disadvantage. The developers’ skills and awareness in the performance area have been abandoned and the tools are not currently available that enable us either to measure application and system performance effectively or, indeed, to tune our systems appropriately.

It would be misleading to suggest that it was only the evolution of the architectures that has led to this situation. Software and systems are critical to the provision of competitive advantage nowadays and even short delays in the delivery of systems can mean the loss of significant market share. The temptation, therefore, is very much to deliver the system and fix the performance later, if necessary, in order not to miss the commercial window of opportunity. This, of course, implies that it is both possible and economic to ‘fix-it-later’ — would that it were always so! To counter the dangers of this extremely risky approach, this paper proposes that it is crucial to engineer performance

into systems and describes some examples where this has been done successfully.

1.2 What is performance engineering?

As has already been suggested, performance engineering (PE) is a life cycle process that helps to assure the performance of a system. It should start as soon as the project starts and continue until the final version of the software is withdrawn from use. The activities can be summarised as follows:

- performance requirements,
- performance risks,
- performance prediction,
- performance management,
- performance measurement.

These do not conform directly to sequential stages of a life cycle, although there is some degree of ordering. There are some dependencies between activities and, as with most life cycles, there is much iteration, both within stages and between stages, and some of the activities will run continuously in parallel.

Figure 1 shows how the activities fit together in a comparatively simple model. The upper boxes show the main activities in bold lines. Dotted lines indicate that the

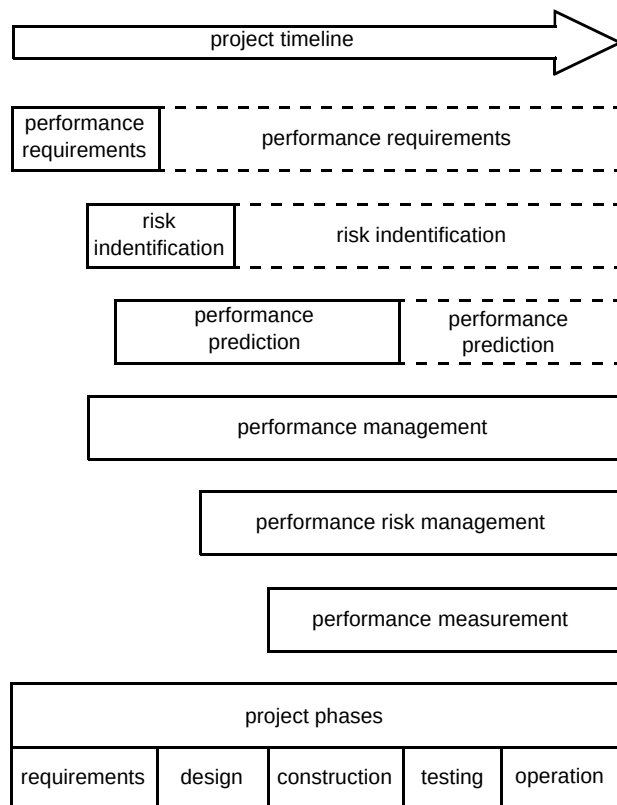


Fig 1 Performance activities.

activity will occur again, perhaps a number of times, during this period. The lower boxes show an approximate fit to project phases. The size of the boxes is not in proportion to the length of the project phase.

2. Performance engineering activities

2.1 Performance requirements

This is an absolutely crucial area that is nearly always neglected or, at the least, lacks appropriate detail. The essence of good performance requirements is to start with the business processes (or workstrings) and to use the business requirements as a way of identifying end-to-end performance measures — including the workload applied to the system and the perceived system delays (user response times, batch run times, event processing times). These can then be broken down in an appropriate way for each element of the business process. It is very important to bear in mind the difference in interpretation that different people put on the term ‘performance’. In a situation where response times have improved dramatically at the expense of usability or throughput at the user interface, it may well be appropriate to describe this as ‘worse’ performance. Involvement with users is crucial whenever requirements are gathered or changes are made; mutual understanding can only benefit the entire process. Gathering of performance requirements is often hide-bound by traditions that take no account of the way in which modern systems use new technologies such as client/server (e.g. all user transactions will be achieved in 2 sec or less. Why? Because that is the target to which we have always worked). Requirements are often based around technology needs and not business needs; this can lead to considerable difficulties in assessing and managing risk. Even the best-expressed set of functional requirements is rarely supported by the equivalent in performance terms — how often do we find (once it is too late!) that ‘*n* transactions per day’ really means ‘*n* transactions per 8-hour working day’.

Often there is a lack of clarity about what exactly a system is going to be used for by which different groups of users, leading to a poor understanding of the user workloads. It can also be equally difficult to build a good understanding of other kinds of workload. Alarm and event handling are typical examples of external stimuli that can also be difficult to quantify. Perhaps a more straightforward area is batch; the problem here may be more in the area of defining appropriate windows for different batch processes and balancing with on-line workloads. Whatever the nature and size of workloads, good information at this early stage is vital for preparing an appropriate set of performance requirements. These can then be tracked throughout design and development, re-assessed at intervals (to ensure the goals remain realistic), and then used for risk identification and management as well as the basis for later testing and measurement.

2.2 *Performance risks*

In an ideal world, there would be enough money and effort to ensure that all aspects of a system's performance would be properly assessed and shown to be satisfactory prior to delivery. This rarely proves to be the case and a method is generally required to enable an appropriate prioritisation of the performance assurance activity to address those areas which are likely to be most at risk of causing a failure to meet business requirements. The risk identification and risk management process depends heavily on there being good data at an early stage in the project. The risk process need not be elaborate — it is probably enough to identify each risk with a description, a likelihood of occurring, and an impact if it were to occur. From these, using weightings that can be varied according to the nature of the project, a priority can be calculated which can be used to focus effort appropriately. Risks should be reviewed on a regular basis and clear plans put in place to manage or control each one. In this way there is a clear link between business need and performance engineering effort applied.

2.3 *Performance prediction*

Too often, the question that asks in advance 'How will the system perform?' is ignored. A typical reply might be that it cannot be predicted, but it can be measured once it is in existence. For some systems this may be appropriate, but in the majority of cases we will find ourselves fighting a losing battle if we rely on a 'fix-it-later' approach to create systems that perform well. In any case, it need not be like that; these days there are a number of methods and tools and approaches that can be adopted to assist in the prediction of performance. In almost all cases there are models that can be created, from simple spreadsheet models to complex simulation models, which can support performance prediction [1]. The most invaluable aid, of course, is the experienced performance analyst who, through years of working in the area, has built a repertoire of knowledge about the behaviour of systems running on particular platforms. However, as these people are not easy to acquire and retain, it is good to have access to appropriate methods and tools. Over the past few years, the Systems Performance group (SPG) at BT Laboratories (BTL) has made extensive use of general-purpose simulation modelling packages, which enable the very rapid development of models. Animation of the models means they can be used to 'walk through' the designs — this greatly improves the communication between users, designers and performance engineers, and enables a check on both the system design and the representation of the system design as implemented in the model. Subsequent exercising of the model using carefully selected input parameters enables the identification of potential trouble spots. The level of sensitivity of the results to changes in the input parameters can be used to reinforce or adapt the risk assessments. The dynamic model will also help to bring out

the effects of interaction between different workloads (as well as multi-streaming of the same workload), something which is often difficult to detect when considering workloads independently or using static models. All of these approaches rely on the ability to gather good data, which is why the requirements phase cannot be neglected, but even at the early stage of projects it is possible to use end-to-end workstrings and associated business requirements as the basis for producing high-level models. These can and should include both automated and manual processes — as more detailed design is achieved, so the models can be refined and revised, perhaps identifying which are the most appropriate manual processes to automate. Again, as the model results can only be as good as the data used to parameterise them, it is essential to continue to work with designers and users, and to draw on the knowledge of experienced engineers to ensure good data. To achieve real value from this approach it is vital that there are feedback loops that enable design decisions to be ratified or altered according to the results of the early predictions. To make this more likely, links between performance engineers and the designers and developers need to be strong. A project-based performance working group, consisting of performance engineering specialists and a key person from each design or development group, which meets regularly to debate issues and assess risks, is a useful approach. This keeps performance to the forefront of the minds of designers and developers and helps to give them ownership of the solution of performance issues.

The SPE approach defined by Smith [2] is strongly favoured and adoption of her seven principles during the design stage will go a long way towards creating responsive systems [3]. The performance prediction phase is very important and can be crucial in influencing the design for better performance but should not be taken as an excuse to 'model to death'. The aim should always be to model to the level of detail which gives results that are accurate enough — in other words, start with simple models and refine only those parts which subsequently need more detail.

2.4 *Measurement and management*

These two activities are very closely related to each other as it is only possible to manage what can be measured. As soon as possible, the measurement process should begin, because it is important for validating models, recognising progress, performance acceptance testing, etc.

What is measured and how? Here again, it is crucial that an end-to-end approach is used; there is a definite need to be able to assess and understand system resource usage but, more importantly, there is a need to be sure that workload throughput can be achieved on an end-to-end basis that will meet business needs. This means that, especially for systems that incorporate non-mainframe boxes, designers and developers have to get much smarter at identifying

which parts of systems need to be measured effectively and to instrument them accordingly. This really cannot be over-emphasised, as the nature of multi-platform, multi-protocol, multi-network systems means that the tracing and timing of transactions and messages is not likely to be confined to a single 'box' which has all the necessary measuring tools.

Equally, current systems often require a different interpretation of their performance characteristics; for example, a system working with the CPU 100% busy may be performing perfectly — the context needs to be understood.

Another crucial aspect of performance management and measurement is the ability to continue the management and measurement beyond design, development and testing and through into operation. For good economic and business reasons it makes sense that the techniques used during design and construction should be transferable to the field for operational monitoring. This requires a good level of co-operation and collaboration between, often, highly disparate parts of an organisation. If more sophisticated measurement and management techniques are to be adopted, there may be significant implications for training to ensure their effectiveness in the longer term.

Another aspect of performance measurement that can prove highly difficult to resolve is the obtaining of test systems of an appropriate size and enough valid data to enable realistic load testing to occur. If this is a serious issue, the increased use of modelling and the evaluation of the ways in which the performance changes as the models are scaled may help to mitigate the lack of real performance test results. However, the value that can be obtained by using these large systems as a training ground for business activity should help to encourage commitment to such test systems. Capture/replay tools that can emulate many users, sometimes from a single replay source, can be effective in creating significant server loads — this can help to overcome the requirement for excessive human intervention.

Not only are the sizes of test systems often inadequate for realistic performance tests, it is often an issue when performance testing requires sole access to a test system for an extended period of time in order to retain full control of the workload. Co-operation with test teams is, therefore, a prerequisite for successful performance measurement — a beneficial by-product can be obtained if test teams can be prevailed upon to run the performance testing process themselves, leaving the performance specialists with more time to analyse results and prepare further tests.

It is important to appreciate how fundamental performance management and measurement is if a system is not only to be effective for the business, but also its effectiveness is to be visible to senior management such that

actions can be taken when appropriate. For highly complex distributed systems, early design considerations should include the possibility of creating a specific component whose responsibility is the management and monitoring of the operational system.

Having looked at the major activities, it is worth mentioning one other aspect — roles and responsibilities — before describing the successful use of some of these activities on recent BT projects.

2.5 Roles and responsibilities

No matter how slick the performance approach is in theory, it can only reap benefit if there is a framework for success in place across the project.

A major factor in ensuring the success of performance work on projects will be the commitment of the overall project managers to the suggested approach and their willingness to give authority to the performance team. Appropriate sign-off for the non-functional requirements of a system should include the signature of the senior performance engineer. At regular project board meetings, review of the status of the performance risks (as well as other risks) for each system component will encourage component managers to assign responsibility for performance within their team. Regular meetings of a performance working group, consisting of performance specialists, designers, developers, testers and operations people can help to ensure that a consistent and coherent monitoring, measuring and management process will be identified and implemented.

3. 'Per-second-pricing' — a case study

Many of the factors that can lead to performance failure in major systems were present in the 'per-second-pricing' project (see Fig 2). BT's move from charging PSTN calls by units to an approach where every call is itemised and separately priced required major enhancements to existing billing systems. Also, a number of new systems were needed, responsibility for which was shared between several BT units as well as major external IT companies. Data volumes were set to rise by an order of magnitude, both in terms of daily data flow and database storage requirements. Protection of revenue streams was crucial to the survival of the company. New file transfer protocols were being developed and used; new, complex data structures were being developed to handle the information for each call record; new file transfer networks were to be built and used; out-sourced developments needed to be successfully integrated with in-house developments, and so on. All these factors, and many more, provided a real challenge to ensure that, from a performance point of view, BT's 'Billing 90s' programme was not a failure.

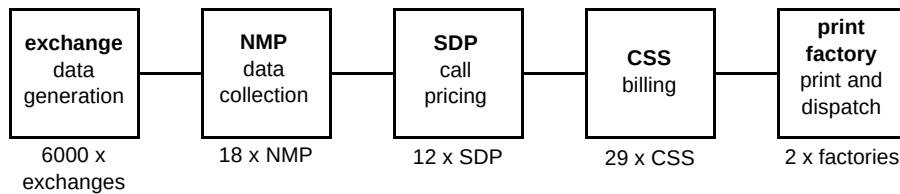


Fig 2 A simple schematic of the 'per-second-pricing' system.

This was therefore, by some way, the most significant opportunity for the demonstration of the benefits of systems performance engineering — but equally there were considerable concerns about the risks of failure.

However, there were some very encouraging aspects of the project. Performance was being addressed from the early stages; it was being addressed from an end-to-end standpoint, not just on a box-by-box basis. It was being properly funded as a major element of the overall programme; it was being invested with the authority necessary to ensure that failures and weaknesses were properly dealt with. All of these were crucial to achieving a positive end result.

Another element that provided a very useful focus was the set of very clear business requirements. It was necessary to transfer a day's priced data into the customer service system (CSS) within the period between about 6 p.m. and 2 a.m. Subsequently (i.e. after 2 a.m.), it was necessary to run the billing programmes and complete them prior to the start of the on-line day at about 8 a.m. These windows applied to all 29 CSS images regardless of the particular volumes of data associated with each image.

It became clear early on that it would be necessary to use these overall requirements as a mechanism for providing target 'windows' for all the component systems, so that there was a realistic chance of achieving the necessary throughput. An early major task was, therefore, to establish an understanding of the data requirements for all the CSS images. This was achieved using a co-operative approach between the SPG and various parts of BT's computing services organisation and proved highly beneficial in establishing a single source of workload information for the CSS mainframes.

Once the specialist performance team of six performance engineers was established, responsibilities for the component systems were assigned and an initial understanding of the 'windows' mentioned above were created; it was not long before the first discrepancy came to light.

Firstly, on the Network Mediation Processor (NMP), there was some confusion about whether the data was to be processed within an 8 or 12 hour period. Secondly, the approach being adopted was to provide a mix of data (i.e. PSTN usage records, inter-network call accounting (INCA) records, exchange statistics, etc) to the NMP throughout the polling window. This did not take into account the need to ensure that all the PSTN usage data could be priced and transferred into CSS before the billing run started. As a result, a period of close co-operative working began between ICL, BT's Worldwide Networks division (the sponsors of the NMP) and the performance team. Processing windows were more clearly agreed and the performance team became more involved with the performance testing that ICL was running. This ensured that not only could the full volumes of data be processed within the full period but that the usage data could be processed and sent to the Service Domain Processor (SDP) within 8 hours. It is important to note that, after an early period when there was some resentment at the involvement of the BT performance team, this subsequently led to a much more positive period of co-operation between all the parties, during which time agreements about ways forward became much easier to reach.

More recent experience in other projects seems to confirm that establishing open and honest dialogue with external suppliers at the earliest possible stage is likely to lead to a much better understanding of performance requirements which can then be implemented to the benefit of all concerned.

One of the most positive aspects of the NMP system was the set of monitoring tools that had been built into the application as part of its development. It cannot be stressed enough that a primary requirement for any system, batch or on-line, is that it should have the ability to monitor and measure itself and report on its own performance. The benefit of using the NMP monitoring was shown to good effect during the end-to-end performance testing.

As has already been mentioned, the NMP was polled for its PSTN usage data by the SDP, which was responsible for pricing the call records before forwarding them to CSS. Again, the performance team worked closely with the developers and VV&T teams to establish clear windows for processing and to monitor the performance testing. The nature of the processing which the SDP performs is highly

CPU-intensive. As files of data were received from the NMP, a pricing engine would do some basic validation checks and then price the call records. Here is an example of a system where, if a processor is being utilised at all, it is working at 100%. During the peak pricing period, it would be expected that all but one (or possibly two) of the processors in the multi-processor machine would be working at 100% — the other CPU(s) would be responsible for handling the communications between the NMP and CSS. This is a prime example of how important it is to understand the context in which a system is operating and not to assume that high CPU utilisation is necessarily indicative of a system that is in need of a performance boost.

In fact, the early performance concerns about the SDP were very little to do with its prime task of pricing calls. There were, however, a number of concerns raised about some of the housekeeping functions that needed to be performed by the SDP. One of these was to load the data from the national charging database (NCDB), the source of all the pricing information, and another was to load 'milestone' files from CSS which contained information about customer options. Pursuit of the performance issues led to the concern that the SDP would need to be taken down every day to load such data and that it could take a number of hours to achieve this. In many of these cases, the value that the performance team provided was to identify the major issues that needed answering by taking an end-to-end view and then to pursue them until a satisfactory understanding or conclusion had been reached.

The NMP and the SDP were both new systems developed for the Billing 90s programme and were designed specifically for the 'per-second-pricing' task. However, the changes that were required for CSS to handle the new approach to billing meant some major development work was required on systems that were already highly complex and established. It was fortunate that the programme recognised the highly significant work that the performance team would be required to do. This meant that the argument to provide a volume test environment that could be dedicated to performance work was recognised and early plans could be made to establish an appropriate performance testing environment and strategy for CSS. When the performance team began their involvement, there had already been significant design work undertaken to identify ways in which the required throughput of the system could be achieved. This included the use of VSAM files to store priced call records rather than the IDMS database. The strategy taken for the performance testing was to capture a day's billing data from an existing CSS database together with its associated performance. This data would then be loaded on to the test environment and run again to provide baseline performance data against which future extrapolation and interpolation could take place.

There was then a process of modelling to establish what the expected volumes of data would be under the new fully itemised billing regime. It was then necessary to generate an appropriate volume of test data that could then be repeatedly run against the various releases of software in order to build a picture of how the applications were performing. At the same time it was necessary to begin to build models to demonstrate the way in which the results could be interpreted to provide information about how the 29 different CSS images would perform, given their hardware environment and likely data volumes. The fact that all the images were of different size with varying numbers of processors, along with disparate amounts of memory and disk space, made this process a highly difficult one. This work was only possible within an environment that was dedicated to the use of the performance team, i.e. where there were no external users or other loads that would distort the performance measures. Even under these conditions, it was a lengthy process to establish such an environment in which regular repeatable performance tests could be undertaken. One of the aspects that simplified the approach was to have sufficient disk space that a disk-to-disk copy of the database and relevant files could be done so that data could be readily restored in order to re-run tests. Without this facility, the process of back-up and restore of data would have prevented the achievement of the amount of testing that was required. Often tests would be run one day and then the data reloaded overnight so that the tests could be repeated the following day — this would have been impossible without the ability to restore data rapidly.

As well as performance testing of the billing components of the system, it was necessary to evaluate the performance of the call-logging component of the system which was responsible for handling incoming data and updating the VSAM file, a highly CPU-intensive task. For this work, much use was made of the Strobe tool to identify the parts of source code that were consuming the majority of the processing. The work done in this area was instrumental in realising a reduction by over 40% in processor demand.

The performance testing that has been described so far has been limited to isolated systems and has not taken account of the performance of the interfaces between systems or of the end-to-end capability. It is natural that testing on a single system would take place before testing on a wider basis.

With a highly complex system such as 'per-second-pricing' it is often not easy to undertake end-to-end performance tests. There are a number of reasons for this — data compatibility between systems can create problems in generating sufficient valid test data, the cost of dedicating appropriate hardware for all the systems involved may be prohibitive, data which will stress one system will not necessarily stress the next system down the line, etc. This may mean that the only way to fully stress test is to have

access to all the full size systems and all the live data. In most cases this is quite unrealistic, so it is necessary to plan carefully what can be actually tested and then model to predict the parts that cannot. In the case of 'per-second-pricing', the early decision had been not to pursue end-to-end stress testing because of the difficulties described above.

However, towards the end of the project it was recognised that it was essential to achieve volume testing of at least the majority of the system, in order to give the business a level of confidence in the system to provide an adequate billing function. To this end, ICL switch emulators were used to deliver virtually full volumes of files to the test NMP. This would process the calls and then pass them on when polled by the SDP. The SDP would then price all the calls and make them available for CSS. Files would then be transferred to CSS and (hopefully) be received by the SD process on CSS. This was as far as the data compatibility issues would allow for successful processing to be achieved.

As it turned out, enormous value was obtained from these volume tests. The first thing that was identified very quickly was that it was not possible to transfer all the files successfully between the NMP and the SDP. After a significant amount of investigation, the problems were resolved by setting the appropriate flow control parameters on the balanced distribution service (BDS) protocol used for transfer. The fact that this problem only surfaced as a result of volume testing and not during normal functional testing, is typical of the kind of benefits that are obtained by doing a rigorous set of performance tests. (It is worth noting that it would have been relatively easy using a dynamic model to have predicted the BDS problems but effort had not been available at that time to construct one.) It was during this phase of testing that the built-in NMP tools had been most useful in tracking down the nature of the performance problems. It was possible to identify the progress of each file through the various processes on the NMP and confirm that it was with the SDP interface that the problems were occurring.

4. Conclusions

A successful project that has used a good performance engineering approach and been successful in delivering a system that performs well (that can be shown to perform well and that can demonstrate that it meets business

needs) is often not recognised as a model to be followed. It may be recognised as a successful project but the performance contribution is transparent. Performance is generally only ever discovered to be a problem once systems go live. Performance specialists are often only called in when a disaster has occurred and, for example, the business cannot bill its largest customer. Even in this far-from-ideal situation, the activities and techniques described above can and will assist in making performance enhancements that will prove effective. It may not be possible to implement a full life cycle approach to each and every emergency but selective use of these techniques against a background of commitment to good performance can bring significant benefit.

References

- 1 'SES/workbench', Scientific and Engineering Software (UK) Ltd, Corinthian Court, 80 Milton Park, Abingdon, Oxon, OX14 4RY.
- 2 Smith C U: 'Performance engineering of software systems,' Addison Wesley (1990).
- 3 SPEED, Performance Engineering Services, PO Box 2640, Santa Fe, New Mexico, 87504.



Nick Chapman graduated from Exeter University in 1977 with a BEd in Mathematics and Philosophy of Education. After teaching for seven years and gaining an MSc in Computing, he joined BT and spent a number of years involved in software tools and methods. He is now the leader of the Systems Performance Group at BT Laboratories and is involved in many major programmes, including OMC, SDH, OMS and PCBS Billing, MCSS and Cambridge. He is a Chartered Engineer and a Member of the British Computer Society.



Lewis Kirby graduated from Imperial College in 1980 with a BSc in Mathematics. After working for Marconi and Scicon in defence, he joined BT in 1986 to work in the area of teletraffic performance. Since then he has been increasingly applying the techniques described in this paper to computer systems. In the past few years he has been leading a small team committed to assuring the performance of BT's billing, including 'per second pricing' and the large flexible billing systems being developed for the future.